# Design Considerations for Integrating Agentic AI with the KNX Building Automation Standard

Moritz Luca Huber
*TUM School of Computation, Information and Technology*
*Technical University of Munich*
Munich, Germany
moritzluca.huber@tum.de

***Abstract*—Agentic AI is increasingly being connected to residential building automation systems such as KNX, a widely used standard in European professional smart homes. This creates a boundary problem: deterministic KNX installations provide physical actuation authority, while LLM-based agents introduce probabilistic behavior and new failure modes. Existing work treats KNX mainly as a data source or control target, and recent smart-home agent research focuses primarily on task success rather than authority governance. This paper derives design considerations for the agent–KNX boundary from a targeted literature review of runtime assurance, mixed-initiative interaction, automation levels, and agentic AI safety, and from comparative analysis grounded in a real-world prototype. We argue for two principles: first, all agent actions should pass through a code-enforced policy layer while KNX retains independent actuation authority; second, autonomous authority should be typed by action class and graduated only through explicit user-mediated transitions, with safety-critical classes barred from autonomous execution. Direct write access without such boundary controls is therefore inappropriate.**

***Index Terms*—agentic AI, KNX, building automation, software engineering for AI, smart home**

## I. Introduction

Agentic AI is being layered on top of existing home automation infrastructure, and KNX (ISO/IEC 14543-3) is a long-established, widely deployed standard for professional building automation in Europe [1]. As Section II details, this integration is under way: channels between AI assistants and KNX installations are commercially established, and multi-step LLM agents are emerging on platforms from which KNX is directly reachable. Design guidance for the boundary between these two technologies, however, is scarce: on one side a deterministic, field-proven fieldbus substrate, on the other a probabilistic agentic layer.

This boundary—specifically, how authority over actuation is partitioned across it and how the agent's authority graduates over time—remains under-treated. Prior work engages KNX as a data source for ML pipelines or as a control target for natural-language interfaces, and the emerging literature on LLM smart-home agents optimizes for task success. To the best of our knowledge, no prior work models a substrate that holds independent, pre-existing actuation authority (Section III).

This paper proposes design considerations for the agent–KNX boundary, derived from a targeted literature review and grounded against a working prototype deployed on a residential KNX installation. Specifically, we address the following research question: *What design considerations apply to the boundary between an agentic AI layer and a KNX building automation substrate?* We decompose this into two sub-questions:

**RQ1:** How should authority over actuation be partitioned between agent and substrate?

**RQ2:** How should the agent's authority to act autonomously be governed and graduated over time?

The remainder of the paper is structured as follows. Section II motivates the problem. Section III positions the work against related literature. Section IV describes our methodology and the reference prototype. Section V develops the two design considerations, answering RQ1 (V-A) and RQ2 (V-B). Section VI concludes.

## II. Motivation

KNX is deterministic and rule-based. Every automation is explicitly engineered, configured per group address and bound to defined triggers; the installation executes what it was programmed to do. This determinism underpins three decades of field-proven reliability, but it also defines the standard's functional limit. A KNX installation cannot interpret a natural-language request, infer intent from context, or respond to situations no integrator anticipated. Requests such as "I'm cold" or "get the house ready for guests" cannot be expressed as predefined programs. LLM-based agents address this limitation, offering intent-driven, conversational control that generalizes beyond programmed behavior. The motivation to combine the two technologies therefore arises from a concrete capability gap. In the residential segment, where systems are purchased optionally and primarily for added comfort, this gap creates demand for integration.

This demand is already being addressed at two levels of capability. The integration surface is commercially mature: the KNX Association presents AI-driven speech processing and voice assistants as an integral part of the KNX smart home [2], and vendors offer corresponding products. The Gira X1 server, for example, in combination with the Gira S1 module, exposes KNX building functions to Amazon Alexa and Google Assistant [3]; its product data sheet lists Apple Siri support from September 2026 [4]. Natural-language control of KNX

is thus commercially established. The agentic level, in contrast, is only now emerging: adjacent smart-home platforms are beginning to offer multi-step LLM agents that interpret open-ended requests, reason over home state, and invoke device actions as tools. Home Assistant, for example, integrates cloud-based and local LLMs as conversation agents with direct device-control authority and Model Context Protocol support [5]. KNX installations are reachable through these channels, since such platforms bridge natively onto the KNX bus. The integration examined in this paper is therefore likely to proceed independently of whether design guidance exists; the open question is under which boundary conditions it takes place.

For KNX specifically, this development is problematic for two reasons. First, KNX provides no native authorization at the fieldbus level [1]. Any device that can write to the bus holds, by default, unrestricted actuation authority. KNX Secure does not close this gap: it authenticates and protects communication and scopes access by keys and topology, but expresses no semantic policy, such as which action classes an authenticated agent may autonomously actuate. A fully authenticated agent retains unrestricted actuation authority the moment it connects; the problem therefore exists prior to, and independently of, any agent misbehavior. Second, what a KNX installation actuates is physical and in part safety-relevant, e.g., doors, locks, alarms, and heating. A probabilistic actor exhibits failure modes—hallucination, prompt injection, multi-step reasoning errors—that have no counterpart in the deterministic substrate [6]. The integration described above connects such an actor to physical and partly irreversible consequences. The capability gap and this risk profile meet at the boundary between the agentic layer and the KNX substrate. This intersection makes explicit design considerations necessary and is the subject of RQ1 and RQ2.

## III. Related Work

Runtime assurance for safety-critical control established the pattern this paper builds on: an unverified, high-performance component is exploited behind code-enforced guardrails while a verified substrate retains control. Sha's Simplex architecture is the canonical formulation [7]. Section V-A develops this mapping; the key difference in our residential setting is a substrate whose authority is inherited rather than purpose-built as a fallback.

The degree of initiative an autonomous system should take has been studied extensively. Horvitz formulates initiative as an expected-utility decision under uncertainty about user goals, and his principles of mixed-initiative interaction lay part of the conceptual foundation for our work [8]. Parasuraman's account of automation as typed and graduated across levels—non-binary, set per function, and consequence-cost driven—provides the basis for treating authority as a typed dimension [9]. The adjustable-autonomy literature moves from static levels to dynamic transfer of control between agent and human during operation [10]. All three predate LLMs and agentic AI, yet supply the vocabulary and building blocks for integrating agentic AI with established standards such as KNX.

In agentic AI safety and runtime enforcement specifically, AgentSpec provides a runtime DSL of trigger, predicate, and enforcement that gates agent tool calls—the closest enforcement mechanism to our prototype [11]. Doshi et al. show that model-based safeguards reduce but cannot eliminate risk, so constraints must move outside the model into code-enforced guardrails [6]. Scerri et al. introduce forbidden-state and forbidden-action constraints that take lexical priority over expected value, establishing that some actions must be hard-excluded rather than traded off [12]. None of these works models a substrate with independent prior authority, and none addresses building automation.

Typed, user-granted permissions for the home also predate LLM agents: He et al. motivate per-capability rather than per-device access control [13], and SmartAuth enforces user-granted permissions on automation apps at the platform level [14]. This work grants authority statically on hub platforms and models neither a probabilistic planner nor a substrate with independent prior authority—the two dimensions Section V adds.

KNX itself, and agentic-AI integration with KNX in particular, remains under-treated—and is therefore the contribution space of this paper. Granzer et al. characterize the security properties of our substrate [1]; the absence of bus-level authorization (Section II) gives a connected agent default actuation authority. Vanus et al. treat KNX as a data source for machine-learning-based occupancy estimation [15], and Hamady et al. introduce natural language processing and LLMs as a control interface to KNX [16].

The work thematically closest to our contribution applies LLM agents to smart-home control, interpreting natural language to plan and execute device actions [17]–[19]. The shared objective across these papers is task success—in essence, "can the agent complete the instruction?" [17], [18]. SAGE foregrounds the absence of hard-coded decision logic as a central feature—the inverse of Section V-A, which treats a code-enforced boundary as a precondition for safe deployment. Even as this line of work turns toward safety, it leaves the boundary itself unaddressed: HomeBench notes that executing invalid instructions risks unsafe actions, yet frames this as instruction-validity accuracy rather than an architectural concern [18]. SmartBench, the most recent and safety-aware of these, evaluates whether an agent can detect anomalous states—including doors left unlocked—but treats this as after-the-fact detection rather than governed authority [19]. To the best of our knowledge, no reviewed LLM smart-home agent or benchmark both partitions actuation authority with class-level exclusions and operates on KNX (Table I).

## IV. Methodology

Our research approach combines a targeted literature review with comparative conceptual analysis. The review was not conducted as a systematic mapping study; it was guided by the paper's research question and the need to relate three bodies of work relevant to the agent–KNX boundary: runtime assurance,

TABLE I
PRIOR WORK COMPARED ALONG THE FOUR ANALYSIS DIMENSIONS OF SECTION IV. NO REVIEWED APPROACH COMBINES AN INHERITED SUBSTRATE AUTHORITY WITH CODE-LEVEL ENFORCEMENT, PER-CLASS TYPING INCLUDING EXCLUSIONS, AND GOVERNED AUTHORITY TRANSITIONS.

| Work | Substrate authority | Enforcement locus | Authority typing | Transition governance |
|---|---|---|---|---|
| Simplex [7] | Purpose-built fallback | Code (decision module) | Uniform | Automatic fallback, fixed rules |
| Mixed-initiative [8] | None modeled | Model (expected utility) | Uniform | Per decision, via dialog |
| Levels of automation [9] | None modeled | Conceptual | Typed per function | Static (design time) |
| Adjustable autonomy [10], [12] | None modeled | Model; hard constraints [12] | Uniform | Dynamic, bidirectional |
| AgentSpec [11] | None (gated tools) | Code (runtime DSL) | Per rule | None (no graduation) |
| Verifiable tool-use safety [6] | None modeled | Code (formal specifications) | Per capability (labels) | None (design-time choice) |
| LLM smart-home agents [17]–[19] | None (passive targets) | Model | Uniform | None |
| KNX ML/NLP integration [15], [16] | KNX, not treated | None | None | None |
| **This paper** | **Inherited (KNX)** | **Code (policy layer)** | **Per class, with exclusions** | **Graduated, user-mediated** |

mixed-initiative and adjustable autonomy, and agentic AI safety.

The prototype developed by the author serves as a grounding case. It is neither empirically evaluated here nor used as evidence that the proposed considerations are universally valid. It rather provides a concrete residential KNX context against which literature-derived concepts can be interpreted. The prototype is deployed on a single KNX installation with $\sim$180 group addresses in a residential family home and has been in continuous operation for approximately four months. Section V references implementation aspects only where they clarify the proposed design considerations.

Literature identification started from seed works that provide central concepts for the analysis: runtime assurance and protected substrates in Sha's Simplex architecture [7], mixed-initiative interaction in Horvitz [8], and recent work on safe tool use for agentic AI by Doshi et al. [6]. Further work was identified through citation following and topic-oriented search in Semantic Scholar, IEEE Xplore, and arXiv. Candidate references were checked against publisher metadata before inclusion. Industry and vendor sources cited in Section II were identified separately through the association and vendor websites; they are used only to establish practical integration context, not to support design considerations.

Papers were included if they provided at least one of the following: (a) an architectural pattern relevant to the agent–substrate boundary, (b) a conceptual framework for human–AI authority, initiative, or autonomy, or (c) recent work on AI/ML integration with KNX, smart homes, or building automation systems. Adjacent work on smart-home access control is treated only insofar as it bears on agentic authority.

The comparative analysis examined prior work along four dimensions derived from the research question: the origin of substrate authority, locus of enforcement, typing of authority, and governance of authority transitions. These dimensions distinguish whether authority is purpose-built or inherited, whether constraints are enforced by the model or code, whether authority is uniform or action-class-specific, and whether authority transitions are static, dynamic, or user-mediated. Table I summarizes the reviewed approaches accordingly. Design considerations were derived where existing concepts required nontrivial translation into the residential agentic-KNX context.

The limitations of this method are that the literature review is targeted rather than systematic and that the prototype represents a single deployment. The resulting design considerations should therefore be read as conceptually grounded guidance, not empirically validated general rules.

## V. DISCUSSION

### A. Partitioning Actuation Authority (RQ1)

A connected agent holds unrestricted actuation authority on the KNX bus by default (Section II). Prompt-level safeguards—in-prompt rules, output filtering, self-checking—reduce the probability of unsafe behavior but cannot eliminate it [6], [11]. In a residential context, residual risk on actions such as opening a door or arming an alarm is unacceptable; such actions must be excluded from autonomous execution by design, not mitigated probabilistically. The constraints must therefore be located outside the model's reach and enforced at code level on the tool-call boundary. The precedent for this is the Simplex architecture, developed by Sha [7]. It combines a verified high-assurance core (HAC) and an unverified high-performance component (HPC), with a decision module arbitrating between them. In our case, the HAC is the KNX substrate plus its pre-existing programmed behavior (for example, automations, scenes, and switch bindings), the HPC is the agentic layer, and the decision module is the policy layer (Fig. 1). The substrate plays the role of Sha's high-assurance core as an inherited, deterministic baseline rather than a formally verified one. The mapping is structural and not exact because our policy layer validates individual proposals under reference-monitor semantics rather than arbitrating between redundant controllers; we inherit Simplex's independent core and graceful degradation, not its arbitration. The HPC enhances and never replaces the HAC, which stays operational even if the HPC fails.

Unlike Sha's HAC, which is purpose-built as a fallback, ours is inherited: the KNX authority pre-exists the agent and was never designed for it. The integrator therefore does not construct the high-assurance core but finds it already deployed. Neither classical Simplex nor generic agent-safety work such as AgentSpec [11] makes this move, because their substrates have no prior authority. AgentSpec gates the agent's actions

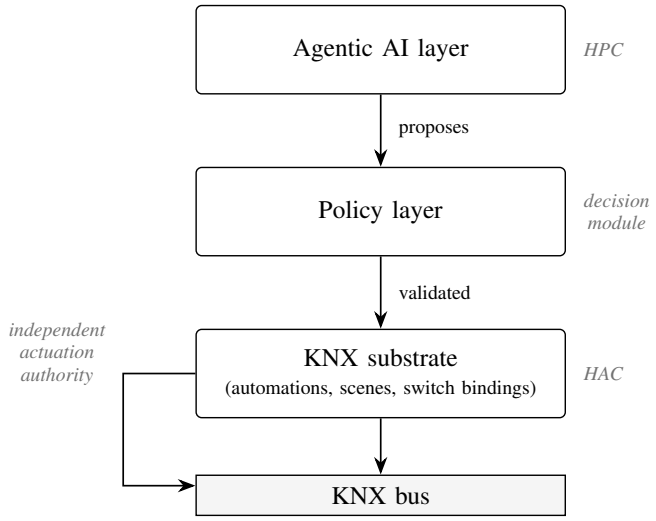



Fig. 1. The agent–KNX boundary. The agentic AI layer (high-performance component) proposes actions that are validated by the policy layer (decision module) before reaching the KNX bus. The KNX substrate (high-assurance core), together with its pre-existing programmed behavior, retains independent actuation authority: a path to the bus that does not pass through the policy layer and remains available even when the agent layer fails.

but models no independent component that retains authority when the agent fails. An alternative would be multiple agents for reliability. However, Sha argues that dividing effort for diversity does not improve reliability unless failure independence holds, and independence generally cannot be proven. For LLM agents in particular, the failure modes of Section II plausibly correlate across instances. Code-level enforcement is therefore the structurally sounder response. The fallback is architectural: when the agent is offline, slow, or suppressed by the policy layer, the home returns to its pre-agent operational mode—graceful degradation. To answer RQ1: the agent holds no direct actuation authority to the bus. All proposals pass through a code-enforced adapter, the policy layer. In the prototype, the policy layer defines declarative rules (for example, DenyAction and SteerAction) evaluated per tool call. For integrators, this implies that an enforceable policy mechanism is a precondition for granting bus write access; prompt-level configuration is not sufficient. Only the policy layer issues writes to the substrate, and all writable group addresses are classified into action classes. Without this invariant, the deny rules would be bypassable and the partition would not hold.

***Design Consideration 1 (Partitioned actuation authority):*** *The agent holds no direct bus credentials and no independent path to the substrate. All proposed actions pass through a code-enforced policy layer, and validated proposals reach the bus as additional commands rather than replacements. The substrate, together with its pre-existing programmed behavior, retains independent actuation authority that remains available when the agent layer fails.*

### *B. Graduating Authority over Time (RQ2)*

In the residential context, trust and experience are not soft goals. Because purchase is optional, a system that does not benefit the user goes unused—or worse, actively degrades comfort, contradicting its purpose. And because automation changes human behavior in ways designers do not anticipate [9], the system must adapt over time. Consequences of actions in a KNX home are not uniform. Switching off lights on request, by accepted automation, and by silent inference are structurally different acts with different consequence profiles. Automation levels should therefore be set per function, using consequence costs as the evaluative criterion; the recommended levels are upper bounds, and high-risk decision automation should stay at the lower end of that range [9]. A single authority setting applied across both light switches and door locks is a category error under this model. Autonomous action is justified when $p(\text{goal} \mid \text{evidence})$ exceeds a threshold set by outcome utilities [8]. Horvitz's expected-utility framing is naturally read as two such thresholds: below the first, do nothing; between them, engage in dialog; above the second, act. This maps onto a graduated ladder—inaction, then suggest or ask, then act autonomously. Early in deployment, the agent's model of the user is poor, so expected utility favors dialog; each user approval shifts the probability upward. Graduation thus reads Horvitz's thresholds as a function of accumulated evidence. The same reading also identifies possible downward evidence: declined suggestions, overridden actions, and policy-layer denials all indicate that the current authority level may be too high. We deliberately do not automate demotion on this evidence. Demotion contracts the agent's authority and, unlike expansion, carries no corresponding hazard; it is therefore available to the user at any time without accumulated evidence. Graduated authority is nonetheless not a one-way ratchet—control moves in both directions, as in the adjustable-autonomy literature [10], but by deliberately asymmetric mechanisms. Parasuraman provides typed but static levels [9]; adjustable autonomy provides dynamic but uniform transfer of control [10]. Our contribution combines what neither covers alone: typed levels with governed transitions and class exclusions. We propose, further, to treat exclusion as the limit case of this model. For actions such as opening a door or arming the alarm, the cost of acting wrongly is severe: the threshold rises above any confidence the agent can attain [8]. Exclusion is therefore enforced as a non-probabilistic deny rule, not as an exception bolted onto the model. This pattern has precedent: Scerri et al.'s users declare forbidden-state and forbidden-action constraints that bind independently of expected-value optimization ( [12], §4.4). The motivation there is the same: the designer lacks a model of the user, learning one requires too many interactions, so the user declares constraints directly [12]. Their users forbid autonomous action to force transfer of control for a class—equivalent to requiring that a safety-critical class must always escalate and never execute autonomously.

A threshold-based escalation rule is itself probabilistic, and

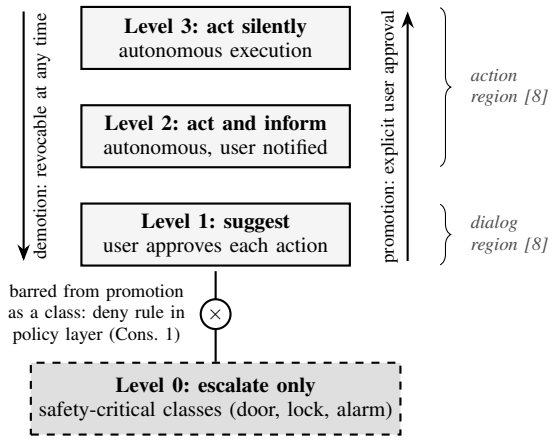



Fig. 2. Graduated authority ladder of Design Consideration 2. Each action class is typed at installation time and assigned a level. Promotion requires explicit user approval; demotion is available to the user at any time without accumulated evidence. Safety-critical classes are pinned at Level 0 and barred from promotion by non-probabilistic deny rules enforced in the policy layer of Consideration 1. Levels 1–3 instantiate Horvitz's dialog and action regions [8].

probabilistic safeguards are insufficient where residual risk is unacceptable [6]. Following Consideration 1, the exclusion is therefore enforced in code, not in the escalation logic: excluded classes are realized as deny rules in the policy layer. This interlock closes the paper's architecture.

Parasuraman's out-of-the-loop findings show that reliable automation degrades monitoring and delays failure detection [9]. For doors and alarms, the damage lies in detection latency, so automating security actions erodes protective attention where it matters most—a second reason for exclusion. The scheme, depicted in Fig. 2, comprises three graduation levels plus the pinned exclusion level: security actions remain at Level 0, while promotion across Levels 1–3 is user-mediated in operation. This is not adaptive automation: adaptive schemes vary level by situation [9], whereas graduation varies it by accumulated trust over time, with the user as gatekeeper. Promotion is user-mediated for a reason. Silent self-expansion of authority is the unanticipated-automation failure mode Parasuraman documents [9], and in an optional-purchase context perceived control is the product. Asking, in Horvitz's framing, is a low-cost dialog action [8]. The action classes themselves are typed by the integrator at installation, extending the structuring already performed when organizing group addresses by function.

***Design Consideration 2 (Typed, graduated, user-mediated authority):*** *Action authority is typed per action class at installation time and graduates over levels ranging from suggestion to autonomous execution, with each promotion subject to explicit user approval and granted authority revocable by the user at any time. Safety-critical classes are barred from autonomous execution as a class; the exclusion is enforced as non-probabilistic deny rules within the policy layer of Consideration 1 rather than within the escalation logic.*

## VI. Conclusion

This paper proposes two design considerations for the agent–substrate boundary. Under Consideration 1, a code-enforced policy layer mediates all agent actuation while the substrate retains independent actuation authority, including in the event of agent failure. Under Consideration 2, action authorities are typed and graduated over time, with explicit, user-mediated transitions and safety-critical classes barred from autonomous execution. For practice, this implies that agentic solutions in the residential sector should not be granted direct write authority over existing KNX infrastructure without comparable boundary controls.

The limitations of this work include the limited deployment time of a single prototype (approximately four months), a targeted rather than systematic literature review reflecting the limited prior work on the agent–substrate boundary specifically, and integration into only a single KNX substrate. Future work may focus on evidence-driven demotion as an extension to user-initiated revocation; a longitudinal deployment could evaluate it via override rate, per-class denial frequency, and time-to-demotion. Further directions are multi-agent (supervisor/specialist) patterns and a closed-loop learning chain.

## References


[1] W. Granzer, F. Praus, and W. Kastner, "Security in building automation systems," *IEEE Trans. Ind. Electron.*, vol. 57, no. 11, pp. 3622–3630, Nov. 2010, doi: 10.1109/TIE.2009.2036033.

[2] KNX Association, "Artificial intelligence in smart devices: How AI is making homes smarter," Mar. 12, 2025. [Online]. Available: https://www.knx.org/knx-en/newsroom/news/news/20250312-Artificial-Intelligence-in-smart-devices-How-AI-is-making-homes-smarter/ [Accessed: Jul. 2, 2026].

[3] Gira Giersiepen GmbH & Co. KG, "Gira X1 – one mini server to control your entire Smart Home." [Online]. Available: https://www.gira.com/en/en/products/smart-home/gira-x1 [Accessed: Jul. 2, 2026].

[4] Gira Giersiepen GmbH & Co. KG, "Gira X1, product data sheet, item no. 209600." [Online]. Available: https://katalog.gira.de/en-INT/datenblatt/209600 [Accessed: Jul. 2, 2026].

[5] P. Schoutsen, "Building the AI-powered local smart home," Home Assistant Blog, Sep. 11, 2025. [Online]. Available: https://www.home-assistant.io/blog/2025/09/11/ai-in-home-assistant/ [Accessed: Jul. 2, 2026].

[6] A. Doshi, Y. Hong, C. Xu, E. Kang, A. Kapravelos, and C. Kästner, "Towards verifiably safe tool use for LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Software Engineering: New Ideas and Emerging Results (ICSE-NIER '26)*, Rio de Janeiro, Brazil, Apr. 2026, arXiv:2601.08012.

[7] L. Sha, "Using simplicity to control complexity," *IEEE Software*, vol. 18, no. 4, pp. 20–28, Jul./Aug. 2001, doi: 10.1109/MS.2001.936213.

[8] E. Horvitz, "Principles of mixed-initiative user interfaces," in *Proc. SIGCHI Conf. Human Factors in Computing Systems (CHI '99)*, Pittsburgh, PA, USA, May 1999, pp. 159–166, doi: 10.1145/302979.303030.

[9] R. Parasuraman, T. B. Sheridan, and C. D. Wickens, "A model for types and levels of human interaction with automation," *IEEE Trans. Syst., Man, Cybern. A, Syst. Humans*, vol. 30, no. 3, pp. 286–297, May 2000, doi: 10.1109/3468.844354.

[10] S. A. Mostafa, M. S. Ahmad, and A. Mustapha, "Adjustable autonomy: A systematic literature review," *Artificial Intelligence Review*, vol. 51, no. 2, pp. 149–186, 2019, doi: 10.1007/s10462-017-9560-8.

[11] H. Wang, C. M. Poskitt, and J. Sun, "AgentSpec: Customizable runtime enforcement for safe and reliable LLM agents," in *Proc. IEEE/ACM 48th Int. Conf. Software Engineering (ICSE '26)*, Rio de Janeiro, Brazil, Apr. 2026, pp. 1–12, arXiv:2503.18666.

[12] P. Scerri, D. V. Pynadath, and M. Tambe, "Towards adjustable autonomy for the real world," *Journal of Artificial Intelligence Research*, vol. 17, pp. 171–228, 2002, doi: 10.1613/jair.1037.

[13] W. He, M. Golla, R. Padhi, J. Ofek, M. Dürmuth, E. Fernandes, and B. Ur, "Rethinking access control and authentication for the home Internet of Things (IoT)," in *Proc. 27th USENIX Security Symp. (USENIX Security '18)*, Baltimore, MD, USA, Aug. 2018, pp. 255–272.

[14] Y. Tian, N. Zhang, Y.-H. Lin, X. Wang, B. Ur, X. Guo, and P. Tague, "SmartAuth: User-centered authorization for the Internet of Things," in *Proc. 26th USENIX Security Symp. (USENIX Security '17)*, Vancouver, BC, Canada, Aug. 2017, pp. 361–378.

[15] J. Vanus, R. Hercik, R. Byrtus, P. Bilik, and J. Koziorek, "Design of a new method for occupancy monitoring in smart home care with autonomous mobile robot within Internet of Things," *Scientific Reports*, vol. 15, no. 1, art. 31767, Aug. 2025, doi: 10.1038/s41598-025-16806-8.

[16] M. Hamady, S. Alsaffar, and L. Canale, "AI-driven user interface for KNX-based home automation systems," in *Conf. Rec. IEEE Industry Applications Society Annual Meeting (IAS)*, Jun. 2025, pp. 1–4, doi: 10.1109/IAS62731.2025.11061550.

[17] D. Rivkin, F. Hogan, A. Feriani, A. Konar, A. Sigal, S. Liu, and G. Dudek, "SAGE: Smart home agent with grounded execution," 2024, arXiv:2311.00772, doi: 10.48550/arXiv.2311.00772.

[18] S. Li, Y. Guo, J. Yao, Z. Liu, and H. Wang, "HomeBench: Evaluating LLMs in smart homes with valid and invalid instructions across single and multiple devices," in *Proc. 63rd Annu. Meeting Assoc. Comput. Linguistics (ACL '25), Vol. 1: Long Papers*, Vienna, Austria, 2025, pp. 12230–12250, doi: 10.18653/v1/2025.acl-long.597.

[19] Q. Zou, Z. Yan, Z. Xu, K. Gao, J. Xiao, and Y. Jiang, "SmartBench: Evaluating LLMs in smart homes with anomalous device states and behavioral contexts," 2026, arXiv:2603.06636.